Laboratirio 2 - Electiva II

Estado	Listo

Laboratorio #02 — Bases de Datos NoSQL con Docker

Asignatura: Electiva II
Cuatrimestre: I 2026
Realizado por: Kevin Córdoba Rivera
Docente: Jorge Ruíz
Tecnologías: HBase · MongoDB · Redis · Docker Compose · Python · Jupyter

---

Introducción

El presente laboratorio tiene como objetivo explorar y comparar tres motores de bases de datos NoSQL ampliamente utilizados en la industria: **Apache HBase**, **MongoDB** y **Redis**. A través de la carga de un dataset real de comercio electrónico con más de **2,6 millones de registros**, se evaluaron aspectos como el modelo de datos, la estrategia de carga, la ejecución de consultas nativas y el rendimiento medido en tiempo de respuesta.

Todo el entorno fue montado con **Docker Compose**, garantizando reproducibilidad entre sistemas operativos (Windows con WSL2 y Linux). Las consultas se implementaron tanto de forma nativa desde cada CLI como mediante scripts Python en cuadernos Jupyter, usando `happybase`, `pymongo` y `redis-py`.

Objetivos

- Desplegar HBase, MongoDB y Redis en contenedores Docker.
- Cargar el dataset de historial de compras de electrónica en los tres motores.
- Ejecutar consultas analíticas equivalentes en cada motor y medir tiempos de respuesta.

- Comparar el modelo de datos, rendimiento y casos de uso de cada tecnología.

Motores de Base de Datos NoSQL

Apache HBase

HBase es una base de datos NoSQL de tipo **columnar distribuida**, inspirada en el modelo BigTable de Google. Está construida sobre HDFS y forma parte del ecosistema Apache Hadoop. Su diseño está orientado a almacenar grandes volúmenes de datos semi-estructurados con baja latencia de lectura/escritura aleatoria.

Característica	Detalle
Modelo de datos	Tablas con *row key*, organizadas en familias de columnas
Escalabilidad	Horizontal, diseñada para clústeres de cientos de nodos
Consistencia	Fuerte (CP en el teorema CAP)
Lenguaje de consulta	No tiene SQL nativo; API Java, Thrift o REST
Casos de uso	Series de tiempo, logs, datos de sensores, historial de eventos
Imagen Docker	`dajobe/hbase`

En este laboratorio se usó el servidor **Thrift** (puerto 9090) para conectar Python. Los datos se organizaron en tres familias de columnas: `event`, `product` y `user`.

MongoDB

MongoDB es la base de datos NoSQL **orientada a documentos** más popular del mundo. Almacena datos como documentos BSON (Binary JSON) con estructuras flexibles y sin esquema fijo. Soporta un lenguaje de consulta rico (MQL) con Aggregation Pipeline, índices secundarios y transacciones ACID multi-documento.

Característica	Detalle
Modelo de datos	Documentos JSON/BSON agrupados en colecciones
Escalabilidad	Horizontal (sharding) y vertical
Consistencia	Configurable (CP por defecto)
Lenguaje de consulta	MQL + Aggregation Pipeline

Característica	Detalle
Casos de uso	Catálogos, APIs REST, análisis en tiempo real
Imagen Docker	`mongo:7.0`

Los datos se cargaron en la colección `purchases` de la base `ecommerce`, con índices en `category_code`, `brand` y `event_time`.

Redis

Redis (*Remote Dictionary Server*) es una base de datos NoSQL **en memoria** de tipo clave-valor, conocida por ser la más rápida del mercado. Opera principalmente en RAM y soporta estructuras de datos avanzadas: strings, hashes, listas, sets, **sorted sets** y streams.

Característica	Detalle
Modelo de datos	Clave-valor con estructuras: hash, list, set, zset, stream
Escalabilidad	Cluster mode, replicación master-replica
Consistencia	Eventual (AP en el teorema CAP)
Persistencia	RDB (snapshots) y AOF (append-only file)
Casos de uso	Caché, sesiones, rankings en tiempo real, colas de mensajes
Imagen Docker	`redis:7.2-alpine`

La estrategia usó **Sorted Sets** para rankings precalculados por categoría, marca e ingresos, permitiendo consultas en tiempo sub-milisegundo.

Dataset Utilizado

Nombre: eCommerce Purchase History from Electronics Store
Fuente: Kaggle — `mkechinov/ecommerce-purchase-history-from-electronics-store`

Atributo	Valor
Archivo	`kz.csv`
Total de registros	2.633.521 filas
Tamaño en disco	~1,3 GB sin comprimir
Período cubierto	Septiembre 2020 – Marzo 2021

Esquema de columnas

Campo	Tipo	Descripción
`event_time`	datetime (UTC)	Marca de tiempo del evento de compra
`order_id`	string	Identificador único del pedido
`product_id`	integer	Identificador del producto
`category_id`	integer	Identificador numérico de la categoría
`category_code`	string (nullable)	Ruta jerárquica (ej: `electronics.smartphone`). ~46% nulos.
`brand`	string (nullable)	Marca del producto
`price`	float	Precio en USD
`user_id`	integer	Identificador del usuario comprador

> ⚠️ **Nota importante:** ~46% de los registros tiene `category_code` nulo. pandas almacena estos valores como `float NaN`, que MongoDB guarda como *BSON double NaN* (no como `null`). Por eso las consultas de categoría deben filtrar con `{ $type: "string" }` en lugar de `{ $ne: null }`.

Configuración del Entorno (Docker)

Requisitos previos

- Windows 10/11 con WSL2 + Docker Desktop, **o** Ubuntu 22.04 con Docker Engine
- Python 3.10 o superior
- 8 GB de RAM mínimo recomendado (HBase consume ~1,5 GB)

docker-compose.yml

```
services:
  hbase:
    image: dajobe/hbase
    container_name: hbase
    ports: ["16010:16010", "9090:9090", "2181:2181"]
```

```
    networks: [labnet]

  mongodb:
    image: mongo:7.0
    container_name: mongodb
    ports: ["27017:27017"]
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: admin123
    volumes: [mongo_data:/data/db]
    networks: [labnet]

  redis:
    image: redis:7.2-alpine
    container_name: redis
    ports: ["6379:6379"]
    command: redis-server --appendonly yes --maxmemory 2gb
    volumes: [redis_data:/data]
    networks: [labnet]
```

Comandos de gestión

```
# Levantar todos los servicios
docker compose up -d

# Verificar estado
docker compose ps

# Iniciar Thrift de HBase (requerido para Python — ejecutar u
na vez)
docker exec -d hbase hbase thrift start
# Esperar ~15 segundos antes de conectar con Python

# Ver logs
docker compose logs <servicio>
```

```
# Detener sin borrar datos
docker compose stop

# Reset total (borra volúmenes)
docker compose down -v
```

Puertos y credenciales

Servicio	Puerto	Credenciales
HBase Thrift API	9090	Sin autenticación
HBase Web UI	16010	Sin autenticación
MongoDB	27017	admin / admin123
Redis	6379	Sin autenticación
JupyterLab	8888	Token en terminal

Entorno Python

```
python -m venv venv
.\\venv\\Scripts\\Activate.ps1   # Windows
source venv/bin/activate       # Linux

pip install -r requirements.txt
```

Biblioteca	Propósito
`happybase`	Cliente HBase vía Thrift
`pymongo`	Cliente oficial MongoDB
`redis`	Cliente Redis
`pandas`	Lectura del CSV y manipulación de datos
`matplotlib` / `seaborn`	Visualización
`jupyterlab`	Entorno de cuadernos interactivos

Carga de Datos en los Tres Motores

Se implementaron scripts Python independientes: `load_hbase.py`, `load_mongodb.py`, `load_redis.py`, y un script unificado `load_data.py`. El CSV se procesó en **chunks de 10.000 registros**.

Estrategia HBase

Tabla `ecommerce` con tres familias de columnas. Row key compuesta: `order_id_product_id`. Uso de `batch()` de happybase para agrupar puts en lotes de 1.000.

```
families = {"event": {}, "product": {}, "user": {}}
conn.create_table("ecommerce", families)

row_key = f"{row['order_id']}_{row['product_id']}".encode()
batch.put(row_key, {
    b"event:time"    : str(row["event_time"]).encode(),
    b"product:cat"   : str(row["category_code"]).encode(),
    b"product:brand" : str(row["brand"]).encode(),
    b"product:price" : str(row["price"]).encode(),
    b"user:id"       : str(row["user_id"]).encode(),
})
```

Estrategia MongoDB

`insert_many(ordered=False)` por chunks. Timestamps convertidos a `datetime` con zona UTC. NaN de pandas convertidos a `None`.

```
chunk["event_time"] = pd.to_datetime(chunk["event_time"], utc
=True, errors="coerce")
docs = chunk.where(pd.notnull(chunk), None).to_dict("record
s")
collection.insert_many(docs, ordered=False)
```

Estrategia Redis

Multi-tipo: cada pedido como `HASH`, más tres **Sorted Sets** precalculados:

- `sales:by_category` → score = cantidad de ventas acumulada por categoría
- `revenue:by_brand` → score = ingresos totales acumulados por marca
- `sales:by_month` → score = cantidad de ventas acumulada por mes

```
pipe.hset(f"order:{order_id}", mapping={...})
pipe.zincrby("sales:by_category", 1, category)
pipe.zincrby("revenue:by_brand", price, brand)
pipe.zincrby("sales:by_month", 1, month)
```

Tiempos de carga

Motor	Registros	Tiempo	Velocidad
MongoDB	2.633.521	~74 s	~35.500 reg/s
Redis	2.633.521	~120–180 s	~15.000–22.000 reg/s
HBase	2.633.521	~25–40 min	~1.100–1.700 reg/s

Consultas Nativas

HBase Shell

```
docker exec -it hbase hbase shell
```

```
list
describe 'ecommerce'
scan 'ecommerce', {LIMIT => 10}

# Filtrar por marca samsung
scan 'ecommerce', {
  FILTER => "SingleColumnValueFilter('product','brand',=,'bin
ary:samsung')",
```

```
  LIMIT => 20
}
```

MongoDB — mongosh

```
docker exec -it mongodb mongosh -u admin -p admin123 --authen
ticationDatabase admin
```

```
use ecommerce

// Q1: Categoría más vendida
db.purchases.aggregate([
  { $match: { category_code: { $type: "string" } } },
  { $group: { _id: "$category_code", total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $limit: 1 }
])
// → [ { _id: 'electronics.smartphone', total: 357581 } ]

// Q2: Marca con más ingresos brutos
db.purchases.aggregate([
  { $match: { brand: { $type: "string" } } },
  { $group: { _id: "$brand", revenue: { $sum: "$price" } } },
  { $sort: { revenue: -1 } },
  { $limit: 1 }
])
// → [ { _id: 'samsung', revenue: 90024615.05 } ]

// Q3: Mes con más ventas
db.purchases.aggregate([
  { $project: { month: { $dateToString:
      { format: "%Y-%m", date: "$event_time", timezone: "UTC"
} } } } },
  { $group: { _id: "$month", total: { $sum: 1 } } },
```

```
  { $sort: { total: -1 } },
  { $limit: 1 }
])
```

Redis CLI

```
docker exec -it redis redis-cli
```

```
# Q1: Categoría más vendida
ZREVRANGE sales:by_category 0 0 WITHSCORES

# Q2: Marca con más ingresos
ZREVRANGE revenue:by_brand 0 0 WITHSCORES

# Q3: Mes con más ventas
ZREVRANGE sales:by_month 0 0 WITHSCORES

# Top 5 categorías
ZREVRANGE sales:by_category 0 4 WITHSCORES

DBSIZE
```

Consultas Python (Jupyter)

Tres notebooks en `queries/`, uno por motor. Cada uno implementa las mismas tres consultas, mide tiempos y genera tablas + gráficos.

#	Consulta	Descripción
Q1	Categoría más vendida	Top 10 por cantidad de ventas
Q2	Marca con más ingresos	Top 10 por ingresos brutos en USD
Q3	Mes con más ventas	Serie temporal mensual en UTC

queries_mongodb.ipynb — Q1

```
pipeline_q1 = [
    {'$match': {'category_code': {'$type': 'string'}}},
    {'$group': {'_id': '$category_code', 'total': {'$sum':
1}}},
    {'$sort':  {'total': -1}},
    {'$limit': 10}
]
t0 = time.time()
result_q1 = list(col.aggregate(pipeline_q1))
elapsed_q1 = time.time() - t0
```

queries_redis.ipynb — Q1

```
t0 = time.time()
raw_q1 = r.zrevrange('sales:by_category', 0, 9, withscores=Tr
ue)
elapsed_q1 = time.time() - t0
df_q1 = pd.DataFrame(raw_q1, columns=['category', 'count'])
```

queries_hbase.ipynb — Q1

```
t0 = time.time()
counter = Counter()
for _key, data in table.scan(columns=[b'product:cat']):
    cat = data.get(b'product:cat', b'unknown').decode(errors
='replace')
    counter[cat] += 1
elapsed_q1 = time.time() - t0
df_q1 = pd.DataFrame(counter.most_common(10), columns=['categ
ory', 'count'])
```

> 📸 *[Insertar capturas de pantalla de los gráficos generados]*

Resultados y Análisis Comparativo

Resultados de las consultas

Consulta	Resultado
Q1 – Categoría más vendida	`electronics.smartphone` — 357.581 ventas
Q2 – Marca con más ingresos	`samsung` — USD 90.024.615
Q3 – Mes con más ventas	`2021-01` (enero 2021)

Tiempos de respuesta por motor

Consulta	MongoDB	Redis	HBase
Q1 – Categoría más vendida	~1–3 s	< 0,001 s	~25–35 s
Q2 – Marca con más ingresos	~1–3 s	< 0,001 s	~25–35 s
Q3 – Mes con más ventas	~2–4 s	< 0,001 s	~25–35 s

> 📸 *[Insertar gráfico comparativo de tiempos de respuesta]*

Comparación de modelos de datos

Aspecto	HBase	MongoDB	Redis
Modelo	Columnar (familias)	Documentos JSON	Clave-valor / estructuras
Esquema	Semi-flexible	Schema-less	Sin esquema
Consultas analíticas	Limitadas (scan + código)	Ricas (MQL + Pipeline)	Limitadas a estructuras precalculadas
Velocidad de consulta	Lenta	Media-Alta	Muy alta (µs)
Velocidad de carga	Lenta	Alta	Media
Persistencia	Sí (HDFS)	Sí (WiredTiger)	Opcional (RDB/AOF)
Escalabilidad	Horizontal nativa	Horizontal (sharding)	Cluster

Análisis por motor

MongoDB demostró el mejor balance entre flexibilidad, velocidad de carga y rendimiento de consulta. El Aggregation Pipeline permite expresar consultas analíticas complejas de forma declarativa. Los índices secundarios redujeron significativamente el tiempo de respuesta.

Redis fue el motor más rápido en consulta con diferencia, respondiendo en microsegundos gracias a los Sorted Sets precalculados. Requiere diseñar las estructuras de datos anticipando las consultas necesarias. Ideal como capa de caché sobre datos ya agregados.

HBase fue el motor más lento tanto en carga como en consulta para este caso de uso. Al no tener lenguaje de consulta nativo rico, las agregaciones requieren full table scans y procesamiento del lado del cliente. Su fortaleza está en escrituras masivas en entornos distribuidos de escala petabyte.

Conclusiones

La realización de este laboratorio permitió explorar en la práctica tres paradigmas distintos del universo NoSQL, evidenciando que **no existe un motor universalmente superior**, sino que cada uno presenta ventajas según el tipo de carga de trabajo.

Sobre los modelos de datos: El modelo de documentos de MongoDB resultó el más natural para representar registros de transacciones. El modelo columnar de HBase es poderoso para series temporales masivas pero su API de bajo nivel requiere más código. Redis obliga a pensar en las consultas antes de diseñar la estructura, lo que produce el rendimiento más alto.

Sobre los tiempos de respuesta: La diferencia fue de varios órdenes de magnitud. Redis respondió en menos de 1 ms, MongoDB en 1–4 segundos y HBase entre 25–35 segundos para el mismo tipo de pregunta. Esto corrobora que Redis es ideal para rankings en tiempo real, MongoDB para consultas flexibles y HBase para ingestión masiva en arquitecturas distribuidas.

Sobre Docker: Docker Compose simplificó enormemente el despliegue, permitiendo reproducir la misma configuración en diferentes sistemas operativos con un único comando. Punto de atención: el servidor Thrift de HBase no inicia automáticamente, requiriendo un paso manual adicional.

Sobre la calidad de los datos: El dataset presentó un desafío real: ~46% de valores nulos en `category_code`. El comportamiento diferente de pandas (`float NaN`), MongoDB (BSON double NaN) y las soluciones requeridas (`$type: "string"`) ilustra la importancia de entender cómo cada tecnología trata los valores faltantes.

Bibliografía

1. Apache Software Foundation. (2024). *Apache HBase Reference Guide*. https://hbase.apache.org/book.html
2. MongoDB, Inc. (2024). *MongoDB Manual v7.0*. https://www.mongodb.com/docs/manual/
3. Redis Ltd. (2024). *Redis Documentation*. https://redis.io/docs/
4. Kechinov, M. (2021). *eCommerce Purchase History from Electronics Store* [Dataset]. Kaggle. https://www.kaggle.com/datasets/mkechinov/ecommerce-purchase-history-from-electronics-store
5. Docker Inc. (2024). *Docker Compose documentation*. https://docs.docker.com/compose/
6. happybase contributors. (2023). *HappyBase documentation*. https://happybase.readthedocs.io/
7. Fowler, M., & Sadalage, P. J. (2012). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional.
8. George, L. (2011). *HBase: The Definitive Guide*. O'Reilly Media.